

GUÍA TÉCNICA · MACHINE LEARNING APLICADO

Encoding de Variables Categóricas

De strings a features para modelos predictivos

Para desarrolladores Python, analistas de datos y estudiantes de ML.

20 técnicas · scikit-learn & category_encoders · casos reales y laboratorio.

Del problema a la práctica

01 • El problema

Por qué los modelos no leen strings y qué es codificar.

02 • Tipos de variable

Nominal, ordinal y binaria: la decisión de fondo.

03 • Las 20 técnicas

De Label a Embeddings, con código y trade-offs.

04 • Elegir bien

Cardinalidad, algoritmo y fugas de datos.

05 • scikit-learn

Pipeline y ColumnTransformer sin leakage.

06 • Casos y lab

Ejemplos reales, experimento y cheat sheet.

1

SECCIÓN 1

El problema de los strings

Un modelo es álgebra: solo entiende números.

Un modelo no sabe qué es "Rojo"

Los algoritmos de ML operan con vectores y matrices. Cualquier columna de texto debe convertirse en números antes de entrenar, o el ajuste falla.

QUÉ BUSCAMOS AL CODIFICAR

- Preservar la información útil de la categoría.
- No inventar orden ni distancias que no existen.
- Controlar cuántas columnas nuevas se generan.
- Evitar filtrar información del target (leakage).

LO QUE ROMPE SIN ENCODING

```
import pandas as pd
from sklearn.linear_model import LogisticRegression

X = df[['color']] # 'Rojo', 'Azul', ...
LogisticRegression().fit(X, y)
# ValueError: could not convert
# string to float: 'Rojo'
```

La idea clave

Codificar (encoding) es traducir categorías a números conservando su significado. No es un paso cosmético: define cuánto puede aprender el modelo.

2

SECCIÓN 2

Nominal vs. ordinal

La naturaleza de la variable manda sobre la técnica.

Tres naturalezas, tres caminos

Nominal

Sin orden natural

- Color, país, método de pago
- Comparar por igualdad, no por <
- One-Hot, Target, Hash, Embeddings

Ordinal

Con orden real

- Bajo < Medio < Alto, talles
- El orden aporta información
- Ordinal Encoding respeta la escala

Binaria

Solo dos valores

- Sí/No, aprobado/rechazado
- Un único bit 0/1
- Label o WOE si es el target

Regla de oro: nunca impongas un orden numérico a una variable nominal. Ese es el error #1 de encoding.

SECCIÓN 3

¿Qué es codificar?

Una misma variable, muchas representaciones posibles.

3

Una variable, cinco transformaciones

La columna "Ciudad" puede convertirse de formas muy distintas. Cada una cambia la dimensionalidad y lo que el modelo puede aprender.

Ciudad	Label	Ordinal	OH-BA	OH-Cba	OH-Ros	Binary	Target
Buenos Aires	0	1	1	0	0	0 0	0.62
Córdoba	1	2	0	1	0	0 1	0.48
Rosario	2	3	0	0	1	1 0	0.55

Label

Un entero por categoría (orden falso).

Ordinal

Enteros con orden definido a mano.

One-Hot

Una columna binaria por ciudad.

Binary

Bits que comprimen las categorías.

Target

Media del target por categoría.

Mismo dato, distinta forma: la elección correcta depende de la cardinalidad y del modelo.

SECCIÓN 4

Las 20 técnicas de encoding

De la más simple a la de estado del arte, con código.

4

El catálogo de un vistazo

Básicas

- 01 Label
- 02 Ordinal
- 03 One-Hot
- 04 Dummy

Compactas

- 05 Binary
- 06 Base-N
- 07 Hash
- 08 Count
- 09 Frequency

Basadas en target

- 10 Target
- 11 Leave-One-Out
- 12 CatBoost
- 13 WOE
- 14 Prob. Ratio

Contraste / avanzadas

- 15 Helmert
- 16 Sum
- 17 Backward
- 18 Polynomial
- 19 Contrast
- 20 Embeddings

Label Encoding

DEFINICIÓN

Asigna a cada categoría un entero consecutivo (0, 1, 2, ...) según orden alfabético o de aparición.

INTUICIÓN

Reemplaza el string por una etiqueta numérica única, sin crear columnas nuevas.

VENTAJAS

- No aumenta la dimensionalidad
- Rápido y muy simple
- Ideal para la variable objetivo (y)

LIMITACIONES

- Introduce un orden falso
- Distancias artificiales entre categorías
- Confunde a modelos lineales y de distancia

Nominal · cualquier card.

ANTES → DESPUÉS

Color	Label
Rojo	0
Azul	1
Verde	2

IMPLEMENTACIÓN

```
from sklearn.preprocessing import LabelEncoder  
le = LabelEncoder()  
y = le.fit_transform(['Rojo', 'Azul', 'Verde'])  
# -> array([1, 0, 2])
```

CARDINALIDAD

Cualquiera (target)

DIMENSIONALIDAD

= 1 columna

ALGORITMOS

Árboles · variable y

02

ENCODING · 02 / 20

Ordinal Encoding

DEFINICIÓN

Mapea categorías con orden real a enteros que respetan ese orden (Bajo < Medio < Alto).

INTUICIÓN

Igual que Label, pero aquí el orden numérico SÍ tiene significado real.

VENTAJAS

- Preserva el orden natural
- Una sola columna
- Compatible con cualquier modelo

LIMITACIONES

- Hay que definir el orden a mano
- Inútil para variables nominales
- Asume distancias iguales entre niveles

Ordinal · baja-media card.

ANTES → DESPUÉS

Nivel	Código
Bajo	1
Medio	2
Alto	3

IMPLEMENTACIÓN



```
from sklearn.preprocessing import OrdinalEncoder
orden = [['Bajo', 'Medio', 'Alto']]
enc = OrdinalEncoder(categories=orden)
X = enc.fit_transform(df[['nivel']])
```

CARDINALIDAD

Baja-media

DIMENSIONALIDAD

= 1 columna

ALGORITMOS

Todos (si hay orden)

03

ENCODING · 03 / 20

One-Hot Encoding

DEFINICIÓN

Crea una columna binaria por categoría; 1 marca la categoría presente y 0 el resto.

INTUICIÓN

Cada categoría vive en su propio eje: sin orden y sin distancias artificiales.

VENTAJAS

- Elimina relaciones falsas de orden
- Excelente en modelos lineales y SVM
- Estándar y muy bien soportado

LIMITACIONES

- Explota la dimensionalidad
- Costoso en memoria y tiempo
- Malo con alta cardinalidad

Nominal · baja card.

ANTES → DESPUÉS

Color	_Rojo	_Azul	_Verde
Rojo	1	0	0
Azul	0	1	0
Verde	0	0	1

IMPLEMENTACIÓN

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(handle_unknown='ignore',
                    sparse_output=False)
X = ohe.fit_transform(df[['color']])
```

CARDINALIDAD

Baja (< 15)

DIMENSIONALIDAD

= k columnas

ALGORITMOS

Lineales, SVM, KNN, NN

La maldición de la dimensionalidad

One-Hot crea una columna por cada categoría distinta. Con variables de alta cardinalidad, esto explota.

EL CASO EXTREMO

Customer_ID

100.000 valores únicos

→ **100.000 columnas nuevas**

- Matriz enorme y casi toda ceros (dispersa)
- Explota el consumo de memoria RAM
- El entrenamiento se vuelve lentísimo
- Los modelos de distancia pierden sentido

COLUMNAS GENERADAS POR CARDINALIDAD

Sexo (2)



2 cols

Provincia (24)



24 cols

Ciudad (2.000)



2.000 cols

Customer_ID (100k)



100.000 cols

Escala logarítmica · One-Hot solo es viable con baja cardinalidad (regla práctica: < 15 categorías).

04

ENCODING · 04 / 20

Dummy Encoding

DEFINICIÓN

One-Hot que elimina una categoría de referencia ($k-1$ columnas) para evitar multicolinealidad.

INTUICIÓN

La categoría omitida queda representada por 'todas las columnas en 0'.

VENTAJAS

- Evita la trampa de las dummy (multicolinealidad)
- Ideal para regresión y modelos estadísticos
- Una columna menos que One-Hot

LIMITACIONES

- Interpretación relativa a la referencia
- Poco relevante en modelos regularizados
- Irrelevante para árboles

Nominal · modelos lineales

ANTES → DESPUÉS

Color	_Azul	_Verde
Rojo	0	0
Azul	1	0
Verde	0	1

IMPLEMENTACIÓN

```
import pandas as pd
X = pd.get_dummies(df['color'], drop_first=True)
# o con sklearn:
OneHotEncoder(drop='first')
```

CARDINALIDAD

Baja

DIMENSIONALIDAD

= $k - 1$ columnas

ALGORITMOS

Regresión lineal /
logística

05

ENCODING · 05 / 20

Binary Encoding

DEFINICIÓN

Convierte cada categoría a un entero y luego a su representación binaria repartida en varias columnas.

INTUICIÓN

Comprime k categorías en aproximadamente $\log_2(k)$ columnas de bits.

VENTAJAS

- Muchas menos columnas que One-Hot
- Bueno para cardinalidad media-alta
- Sin un orden fuerte implícito

LIMITACIONES

- Menos interpretable
- Puede mezclar categorías por bits compartidos
- No siempre mejora la métrica

Nominal · card. media-alta

ANTES → DESPUÉS

Cat	int	b2	b1	b0
A	1	0	0	1
B	2	0	1	0
C	3	0	1	1

IMPLEMENTACIÓN



```
from category_encoders import BinaryEncoder
enc = BinaryEncoder(cols=['ciudad'])
X = enc.fit_transform(df)
```

CARDINALIDAD
Media-alta

DIMENSIONALIDAD
 $\approx \log_2(k)$ cols

ALGORITMOS
Árboles, boosting

06

ENCODING · 06 / 20

Base-N Encoding

DEFINICIÓN

Generaliza Binary Encoding: representa el entero en base N (Binary es simplemente base 2).

INTUICIÓN

Con base mayor usás menos columnas, pero cada una toma más valores posibles.

VENTAJAS

- Control fino entre columnas e información
- Más compacto que One-Hot
- Flexible según la cardinalidad

LIMITACIONES

- La base óptima no es obvia
- Menos interpretable
- Poco usado en la práctica

Nominal · alta card.

ANTES → DESPUÉS

Base	Columnas (k = 1000)
2 (Binary)	10
10	3
1000 (Ordinal)	1

IMPLEMENTACIÓN

```
from category_encoders import BaseNEncoder
enc = BaseNEncoder(cols=['ciudad'], base=4)
X = enc.fit_transform(df)
```

CARDINALIDAD
Alta

DIMENSIONALIDAD
 $\approx \log_N(k)$ cols

ALGORITMOS
Árboles, boosting

Hash Encoding

DEFINICIÓN

Aplica una función hash a la categoría y la mapea a un número fijo de columnas (feature hashing).

INTUICIÓN

Sin diccionario: la categoría se 'tira' a un bucket con una fórmula determinista.

VENTAJAS

- Memoria constante y fija
- Ideal para streaming y online
- Escala a millones de categorías

LIMITACIONES

- Colisiones: dos categorías → mismo bucket
- No reversible / poco interpretable
- `n_components` es un trade-off

Nominal · card. muy alta

ANTES → DESPUÉS

Categoría	hash → columna
Argentina	col_3
Brasil	col_7
Chile	col_3 ⚠

IMPLEMENTACIÓN

```
from category_encoders import HashingEncoder
enc = HashingEncoder(cols=['pais'],
                      n_components=8)
X = enc.fit_transform(df)
```

CARDINALIDAD

Muy alta

DIMENSIONALIDAD

= n fijo

ALGORITMOS

Lineales, NLP, online



ENCODING · 08 / 20

Count Encoding

DEFINICIÓN

Reemplaza cada categoría por el número de veces que aparece en el dataset (su frecuencia absoluta).

INTUICIÓN

Categorías frecuentes reciben valores altos; las raras, valores bajos.

VENTAJAS

- Una sola columna
- Captura la popularidad de la categoría
- Bueno para alta cardinalidad

LIMITACIONES

- Categorías con igual frecuencia colisionan
- Sensible al tamaño del dataset
- Puede no relacionarse con el target

Nominal · alta card.

ANTES → DESPUÉS

País	Count
Argentina	5000
Brasil	3200
Uruguay	150

IMPLEMENTACIÓN



```
from category_encoders import CountEncoder  
enc = CountEncoder(cols=['pais'])  
X = enc.fit_transform(df)
```

CARDINALIDAD
Alta

DIMENSIONALIDAD
= 1 columna

ALGORITMOS
Árboles, boosting

09

ENCODING · 09 / 20

Frequency Encoding

DEFINICIÓN

Como Count, pero normaliza por el total: usa la proporción o frecuencia relativa de cada categoría.

INTUICIÓN

Argentina = 0.25 significa que representa el 25 % de las filas del dataset.

VENTAJAS

- Escala independiente del tamaño
- Una columna, muy compacto
- Robusto para alta cardinalidad

LIMITACIONES

- Colisiones si comparten frecuencia
- No utiliza el target
- Frecuencias raras ≈ 0 se confunden

Nominal · alta card.

ANTES → DESPUÉS

País	Freq
Argentina	0.250
Brasil	0.160
Uruguay	0.007

IMPLEMENTACIÓN



```
freq = df['pais'].value_counts(normalize=True)
df['pais_freq'] = df['pais'].map(freq)
```

CARDINALIDAD

Alta

DIMENSIONALIDAD

= 1 columna

ALGORITMOS

Árboles, boosting

Target Encoding

DEFINICIÓN

Reemplaza cada categoría por el promedio del target para esa categoría (media condicional).

INTUICIÓN

Ciudad → tasa de compra histórica de esa ciudad. Codifica la relación con la etiqueta.

VENTAJAS

- Muy potente y compacto (1 columna)
- Reduce la dimensionalidad drásticamente
- Ideal para alta cardinalidad

LIMITACIONES

- Riesgo alto de DATA LEAKAGE
- Overfitting en categorías raras
- Necesita suavizado y validación cruzada

Nominal · alta card.

ANTES → DESPUÉS

Ciudad	Target medio
Buenos Aires	0.62
Córdoba	0.48
Rosario	0.55

IMPLEMENTACIÓN



```
from sklearn.preprocessing import TargetEncoder
enc = TargetEncoder(smooth='auto', cv=5)
# fit SOLO con train:
X = enc.fit_transform(X_train, y_train)
```

CARDINALIDAD

Alta

DIMENSIONALIDAD

= 1 columna

ALGORITMOS

Boosting, lineales

Leave-One-Out Encoding

Nominal · alta card.

DEFINICIÓN

Target Encoding que, para cada fila, excluye su propio target al calcular la media de la categoría.

INTUICIÓN

Evita que la fila 'se mire a sí misma' → reduce leakage y overfitting.

VENTAJAS

- Menos overfitting que Target puro
- Una sola columna
- Bueno con categorías repetidas

LIMITACIONES

- Aún requiere fit solo en train
- Ruidoso en categorías raras
- Mayor costo de cómputo

ANTES → DESPUÉS

Fila	Media sin la fila i
fila i	$(\sum y - y_i) / (n - 1)$

IMPLEMENTACIÓN



```
from category_encoders import LeaveOneOutEncoder
enc = LeaveOneOutEncoder(cols=['ciudad'])
X = enc.fit_transform(X_train, y_train)
```

CARDINALIDAD
Alta

DIMENSIONALIDAD
= 1 columna

ALGORITMOS
Boosting, lineales

CatBoost Encoding

DEFINICIÓN

Target Encoding 'ordenado': usa solo las filas anteriores (en orden aleatorio) para calcular la media.

INTUICIÓN

Simula datos del pasado: cada fila se codifica sin ver su propio futuro → cero leakage.

VENTAJAS

- Evita el leakage por diseño
- Muy robusto en producción
- Base del algoritmo CatBoost

LIMITACIONES

- Depende del orden de permutación
- Menos intuitivo de explicar
- Rinde mejor dentro de CatBoost

Nominal · alta card.

ANTES → DESPUÉS

Idea	Ventana usada
fila i	filas 1 ... i - 1
fila 1	prior global

IMPLEMENTACIÓN

```
from category_encoders import CatBoostEncoder
enc = CatBoostEncoder(cols=['ciudad'])
X = enc.fit_transform(X_train, y_train)
```

CARDINALIDAD
Alta

DIMENSIONALIDAD
= 1 columna

ALGORITMOS
CatBoost, boosting

DEFINICIÓN

Logaritmo del cociente entre la proporción de positivos y negativos dentro de cada categoría.

INTUICIÓN

Mide cuánta 'evidencia' aporta cada categoría hacia la clase objetivo. $WOE = \ln(\%buenos / \%malos)$.

VENTAJAS

- Estándar en scoring y riesgo crediticio
- Escala monótona y lineal con el logit
- Maneja muy bien el target binario

LIMITACIONES

- Solo para target binario
- Indefinido si hay 0 positivos o negativos
- Riesgo de leakage sin validación cruzada

ANTES → DESPUÉS

Categoría	WOE
Grupo A	+0.84
Grupo B	-0.41

IMPLEMENTACIÓN

```
from category_encoders import WOEEncoder
enc = WOEEncoder(cols=['segmento'])
X = enc.fit_transform(X_train, y_train)
```

CARDINALIDAD
Media-alta

DIMENSIONALIDAD
= 1 columna

ALGORITMOS
Reg. logística (scorecards)

Probability Ratio Encoding

Binario

DEFINICIÓN

Reemplaza la categoría por el cociente $P(y=1) / P(y=0)$ calculado dentro de esa categoría.

INTUICIÓN

Cuántas veces más probable es la clase positiva que la negativa en ese grupo. Es la base de WOE.

VENTAJAS

- Simple e interpretable
- Una sola columna
- Relacionado con WOE (que es su logaritmo)

LIMITACIONES

- Solo para target binario
- División por cero si $P(0) = 0$
- Sensible a categorías raras

ANTES → DESPUÉS

Categoría	P_1 / P_0
Grupo A	2.30
Grupo B	0.66

IMPLEMENTACIÓN

```
p = df.groupby('cat')['y'].mean()
ratio = p / (1 - p)
df['cat_pr'] = df['cat'].map(ratio)
```

CARDINALIDAD
Media

DIMENSIONALIDAD
= 1 columna

ALGORITMOS
Reg. logística

Helmert Encoding

DEFINICIÓN

Codificación de contraste que compara cada nivel con la media de los niveles que le siguen.

INTUICIÓN

Un esquema estadístico para ANOVA y regresión inferencial, no para 'medir distancias' en ML.

VENTAJAS

- Contrastes interpretables en modelos lineales
- Útil en estadística inferencial
- Incluido en `category_encoders`

LIMITACIONES

- Sin ventaja para ML predictivo
- Genera $k - 1$ columnas
- Poco intuitivo

Contraste · estadístico

CONCEPTO

Compara nivel i vs. $\text{media}(\text{niveles } i+1 \dots k)$

IMPLEMENTACIÓN

```
from category_encoders import HelmertEncoder
enc = HelmertEncoder(cols=['grupo'])
X = enc.fit_transform(df)
```

CARDINALIDAD

Baja

DIMENSIONALIDAD

= $k - 1$ cols

ALGORITMOS

Modelos estadísticos

Sum Encoding

Contraste · estadístico

DEFINICIÓN

Contraste que compara la media de cada nivel con la media global (codificación por desviación).

INTUICIÓN

Los coeficientes suman cero; sirve para interpretar el efecto de cada nivel frente al promedio.

VENTAJAS

- Interpretación de efectos vs. la media
- Estándar en ANOVA
- Disponible en `category_encoders`

LIMITACIONES

- No mejora la predicción
- Genera $k - 1$ columnas
- Nicho puramente estadístico

CONCEPTO

Compara cada nivel vs. la media global

IMPLEMENTACIÓN

```
from category_encoders import SumEncoder
enc = SumEncoder(cols=['grupo'])
X = enc.fit_transform(df)
```

CARDINALIDAD

Baja

DIMENSIONALIDAD

= $k - 1$ cols

ALGORITMOS

Modelos estadísticos

Backward Difference Encoding

Ordinal · estadístico

DEFINICIÓN

Contraste que compara cada nivel con el nivel inmediatamente anterior de la escala.

INTUICIÓN

Pensado para variables ordinales: mide el 'salto' entre niveles consecutivos.

VENTAJAS

- Interpretación por escalones (ordinales)
- Contrastes claros entre niveles vecinos
- Base estadística sólida

LIMITACIONES

- Solo tiene sentido si existe orden
- Genera $k - 1$ columnas
- No aporta a árboles ni boosting

CONCEPTO

Compara nivel i vs. nivel $i - 1$

IMPLEMENTACIÓN

```
from category_encoders import BackwardDifferenceEncoder
enc = BackwardDifferenceEncoder(cols=['nivel'])
X = enc.fit_transform(df)
```

CARDINALIDAD

Baja (ordinal)

DIMENSIONALIDAD

= $k - 1$ cols

ALGORITMOS

Modelos estadísticos

Polynomial Encoding

DEFINICIÓN

Contraste que ajusta tendencias (lineal, cuadrática, cúbica...) sobre niveles ordinales equiespaciados.

INTUICIÓN

Captura si el efecto crece de forma lineal o curvada a lo largo de los niveles.

VENTAJAS

- Detecta tendencias en variables ordinales
- Útil en diseño experimental
- Compacto para pocos niveles

LIMITACIONES

- Asume niveles equiespaciados
- Experimental para ML predictivo
- Genera $k - 1$ columnas

Ordinal · experimental

CONCEPTO

Ajusta tendencia lineal / cuadrática / ...

IMPLEMENTACIÓN

```
from category_encoders import PolynomialEncoder
enc = PolynomialEncoder(cols=['nivel'])
X = enc.fit_transform(df)
```

CARDINALIDAD

Baja (ordinal)

DIMENSIONALIDAD

= $k - 1$ cols

ALGORITMOS

Estadística / experimental

Contrast Encoding (familia)

Contraste · panorama

DEFINICIÓN

Familia de codificaciones estadísticas (Helmert, Sum, Backward, Polynomial) basadas en contrastes.

INTUICIÓN

No convierten a número para medir distancias: definen comparaciones entre grupos para modelos lineales.

VENTAJAS

- Interpretabilidad estadística
- Estándar en R y statsmodels
- Control de qué grupos comparás

LIMITACIONES

- Sin ventaja predictiva en ML moderno
- Todas generan $k - 1$ columnas
- Casi ausentes en pipelines de producción

ANTES → DESPUÉS

Miembro	Compara con...
Helmert	niveles siguientes
Sum	media global
Backward	nivel anterior
Polynomial	tendencia

EN UNA FRASE

No convierten a número para medir distancias: definen comparaciones entre grupos para modelos lineales.

CARDINALIDAD

Baja

DIMENSIONALIDAD

= $k - 1$ cols

ALGORITMOS

Modelos lineales /
estadísticos

DEFINICIÓN

Representa cada categoría como un vector denso de números reales aprendido durante el entrenamiento.

INTUICIÓN

Categorías parecidas terminan cerca en el espacio vectorial, igual que en word2vec.

VENTAJAS

- Escala a millones de categorías
- Aprende relaciones semánticas
- Estado del arte en DL y recomendadores

LIMITACIONES

- Requiere red neuronal y muchos datos
- No es interpretable directamente
- Más complejo de implementar

Nominal · card. extrema

ANTES → DESPUÉS

Cliente	Vector aprendido
#1	[0.32, -0.15, 0.87]
#2	[-0.04, 0.71, 0.22]

IMPLEMENTACIÓN

```
import torch.nn as nn
emb = nn.Embedding(num_categorías, 8)
vec = emb(indices)  # (batch, 8)
```

CARDINALIDAD

Extrema (10^5 - 10^6)

DIMENSIONALIDAD

= d (p.ej. 8-256)

ALGORITMOS

Deep Learning, NLP,
RecSys

SECCIÓN 5

Tabla comparativa

Las técnicas clave, enfrentadas por criterio.

5

Cómo se enfrentan las técnicas clave

Técnica	Tipo	Cardinalidad	Columnas	Leakage	Mejor con
Label	Nominal	Cualquiera	1	—	Target / árboles
Ordinal	Ordinal	Baja-media	1	—	Todos (con orden)
One-Hot	Nominal	Baja (<15)	k	—	Lineales, SVM, KNN
Binary	Nominal	Media-alta	$\approx \log_2(k)$	—	Árboles, boosting
Hash	Nominal	Muy alta	n fijo	—	Streaming, NLP
Count / Freq	Nominal	Alta	1	bajo	Árboles, boosting
Target	Nominal	Alta	1	ALTO	Boosting, lineales
CatBoost	Nominal	Alta	1	—	CatBoost, boosting
WOE	Binario	Media-alta	1	medio	Reg. logística
Embeddings	Nominal	Extrema	d	—	Deep Learning

La columna Leakage es la que más cuesta caro en producción: cualquier técnica basada en target exige validación cruzada.

6

SECCIÓN 6

Encoding por algoritmo

El mejor encoding depende del modelo que lo consume.

Qué prefiere cada familia de modelos

Modelos lineales / SVM

Regresión lineal y logística, SVM, KNN

- One-Hot / Dummy para nominales
- Escalar variables numéricas
- WOE en scoring binario

Árboles y boosting

Random Forest, XGBoost, LightGBM

- Ordinal / Label bastan
- Target o CatBoost en alta card.
- One-Hot suele sobrar

CatBoost

Boosting con soporte nativo

- Pasar columnas categóricas crudas
- Codificación ordenada interna
- Cero leakage por diseño

Redes neuronales

MLP, Deep Learning, RecSys

- Embeddings para alta cardinalidad
- One-Hot solo si card. es baja
- Relaciones semánticas aprendidas

SECCIÓN 7

Baja vs. alta cardinalidad

El número de categorías decide la estrategia.



Pocas o muchas categorías

Baja cardinalidad

< 15 categorías

- One-Hot / Dummy sin miedo
- Ordinal si hay orden
- Columnas manejables
- Ej.: sexo, país, día de la semana

Media cardinalidad

15 - 50 categorías

- Binary o Base-N
- Target con validación cruzada
- Evitar One-Hot puro
- Ej.: provincia, categoría de producto

Alta / extrema

> 50 · miles · millones

- Target, CatBoost, Hash
- Embeddings si hay red neuronal
- Nunca One-Hot
- Ej.: código postal, user_id, SKU

8

SECCIÓN 8

Implementación en scikit-learn

Pipeline y ColumnTransformer, sin fugas de datos.

Un pipeline reproducible

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (
    OneHotEncoder, StandardScaler)
from sklearn.linear_model import LogisticRegression

pre = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'),
     ['color', 'ciudad']),
    ('num', StandardScaler(), ['edad', 'ingreso']),
])
model = Pipeline([('pre', pre),
                  ('clf', LogisticRegression(max_iter=1000))])
model.fit(X_train, y_train)  # fit SOLO en train
```

POR QUÉ UN PIPELINE

- Encapsula preprocesado + modelo en un objeto.
- El fit aprende solo del set de entrenamiento.
- Se aplica idéntico a test y a producción.
- `handle_unknown='ignore'` evita romper con categorías nuevas.

REGLA DE ORO

Ajustá el encoder únicamente con datos de entrenamiento y transformá el resto. Ese único hábito elimina la mayoría de las fugas de datos.

9

SECCIÓN 9

La librería `category_encoders`

Todas las técnicas avanzadas con la API de scikit-learn.

Un ecosistema compatible con sklearn

```
pip install category_encoders

import category_encoders as ce

enc = ce.TargetEncoder(cols=['ciudad'],
                        smoothing=10)
X_tr = enc.fit_transform(X_train, y_train)
X_te = enc.transform(X_test)

# misma API para BinaryEncoder,
# HashingEncoder, WOEEncoder, CatBoost...
```

QUÉ APORTA

- fit / transform como cualquier transformer.
- Encaja directo en Pipeline y ColumnTransformer.
- Binary, Hash, Target, WOE, CatBoost, LOO y más.
- Devuelve DataFrames con nombres de columna claros.

CUÁNDO USARLA

- Alta cardinalidad donde One-Hot no escala.
- Necesitás Target/CatBoost con smoothing incorporado.
- Querés experimentar varias técnicas con una sola API.

10

SECCIÓN 10

Errores comunes y data leakage

Los detalles que arruinan un modelo en producción.

El error que infla tus métricas

Data leakage es filtrar información del target (o del futuro) hacia las features. El modelo parece brillante en validación y fracasa en producción.

BIEN: FIT SOLO EN TRAIN

```
enc.fit(X_train, y_train)
X_train = enc.transform(X_train)
X_test = enc.transform(X_test)
# el test nunca influye en el encoder
```

MAL: FIT SOBRE TODO EL DATASET

```
enc.fit(X_full, y_full) # LEAKAGE
# la media de target ya vio el test
```

LOS ERRORES MÁS FRECUENTES

Imponer orden a lo nominal

Label sobre 'ciudad' inventa que Rosario > Córdoba.

Target sin validación cruzada

Memoriza el target de categorías raras (overfitting).

Codificar antes de dividir

El split debe ir antes del fit del encoder.

Ignorar categorías nuevas

Sin handle_unknown, producción se rompe.

One-Hot en alta cardinalidad

Miles de columnas dispersas e inútiles.

11

SECCIÓN 11

Casos reales

Cómo se codifican datasets clásicos de ML.

Cuatro datasets, cuatro decisiones

Titanic

Supervivencia

- Sex → Label (binaria)
- Embarked → One-Hot (3 cats)
- Pclass → Ordinal (1<2<3)

Customer Churn

Clasificación

- Contract → One-Hot
- PaymentMethod → One-Hot
- Country → Target / CatBoost

House Prices

Regresión

- Neighborhood → Target
- Quality → Ordinal
- MSZoning → One-Hot

Adult Income

Clasificación

- Education → Ordinal
- Occupation → Target
- Native-country → Hash / Freq

12

SECCIONES 12 · 13 · 14

Laboratorio y guías finales

Comparación práctica · árbol de decisión · cheat sheet

El experimento: Customer Churn

Un mismo dataset, tres modelos, tres estrategias de encoding. Objetivo: ver cómo la codificación cambia el resultado.

VARIABLES CATEGÓRICAS DEL DATASET

Country	~50 categorías	alta card.
Contract	3 categorías	baja card.
Gender	2 categorías	baja card.
PaymentMethod	4 categorías	baja card.

LOS TRES CONTENDIENTES

Modelo 1

Logistic Regression

→ One-Hot Encoding

Modelo 2

Random Forest

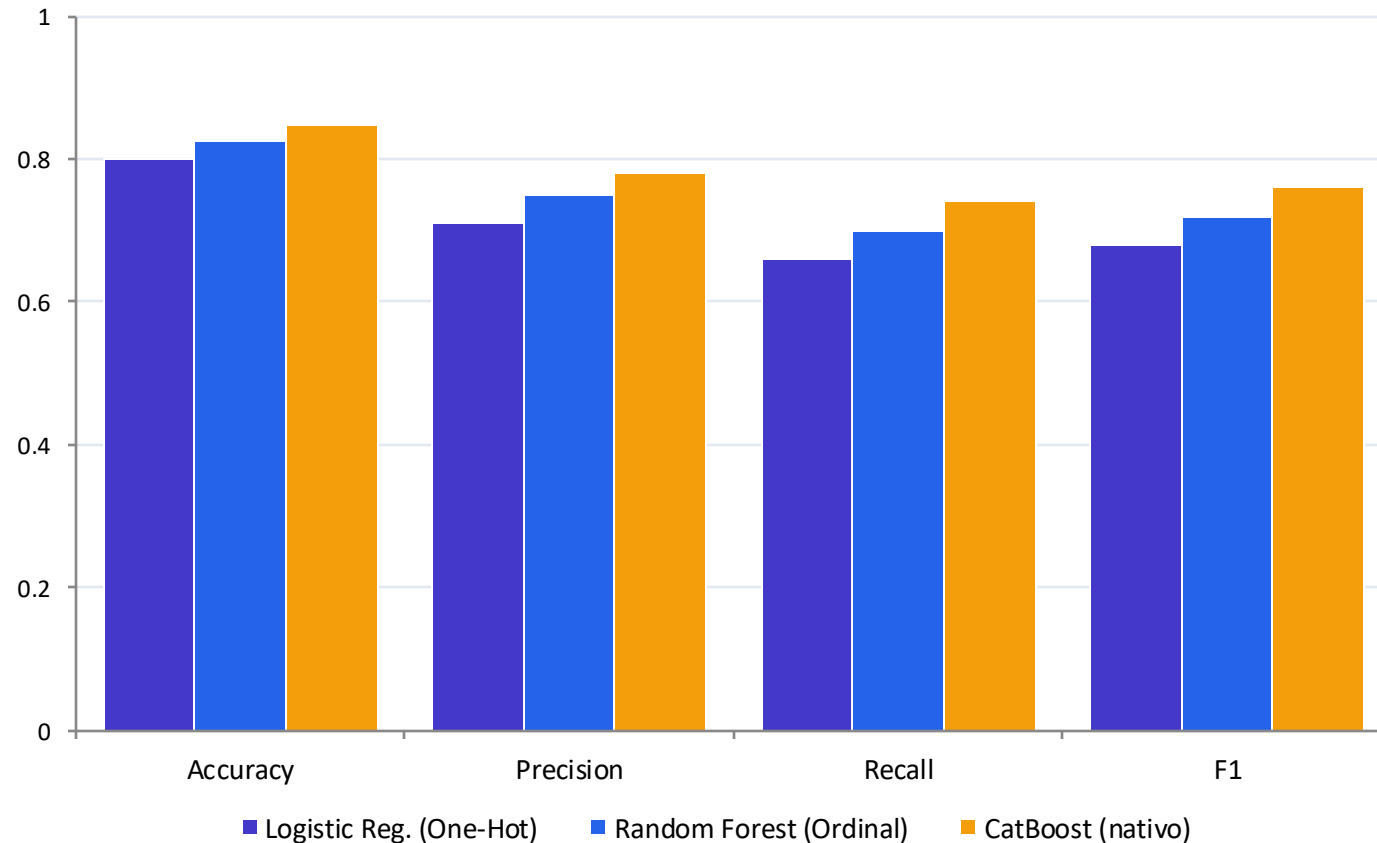
→ Ordinal / One-Hot

Modelo 3

CatBoost

→ Features categóricas nativas

Cómo cambian las métricas

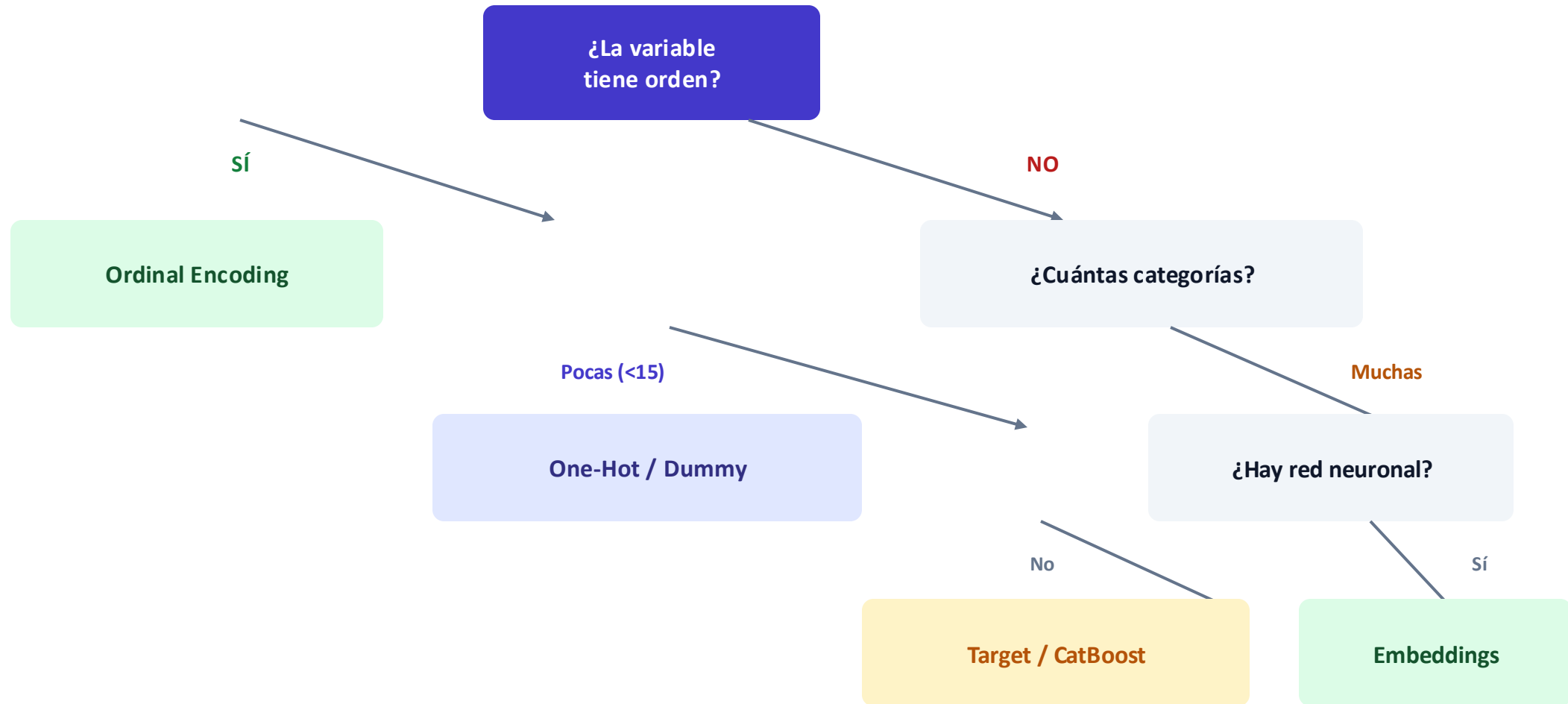


Lectura del experimento

- El manejo nativo de CatBoost gana en las cuatro métricas.
- Random Forest con Ordinal supera a la logística: los árboles no sufren el orden.
- La logística depende de One-Hot; con alta cardinalidad se queda corta.
- Mismo dato, distinto encoding → hasta 4-5 puntos de F1.

Cifras ilustrativas de un pipeline típico de churn.

Cómo elegir tu encoding



Tu variable + tu algoritmo → tu encoding

Tipo de variable	Cardinalidad	Algoritmo	Encoding recomendado
Nominal	Baja (< 15)	Lineal / SVM / KNN	One-Hot
Nominal	Baja (< 15)	Árboles / Boosting	Ordinal • One-Hot
Nominal	Alta (> 50)	Boosting / Lineal	Target • CatBoost
Nominal	Alta (> 50)	Streaming / online	Hash Encoding
Nominal	Extrema (10^5+)	Deep Learning	Embeddings
Ordinal	Cualquiera	Todos	Ordinal Encoding
Binaria (target)	Media-alta	Reg. logística / scoring	WOE Encoding
Identificador	Extrema	Casi cualquiera	Eliminar / Embeddings

Ante la duda: One-Hot si son pocas categorías, Target/CatBoost si son muchas, y Embeddings cuando entrás en Deep Learning.

EN RESUMEN

Codificar bien es la mitad del modelo

- Mirá primero la naturaleza de la variable: nominal, ordinal o binaria.
- Dejá que la cardinalidad y el algoritmo elijan la técnica.
- Ajustá los encoders solo en train: cero data leakage.
- Para alta cardinalidad: Target, CatBoost, Hash o Embeddings.